

Problem A. Aibohphobia

If a character x appears in the string exactly once, the answer is **YES**: we can put it first and then put the rest of the characters in any order. From now on, we assume that every character in the string will appear at least twice.

If all the characters in the string are equal, then $r_1 r_2$ will definitely be a palindrome, so the answer is **NO**.

If there are exactly two different characters in the string, the answer is also **NO**. To show this, let's look at the second occurrence of the character r_1 in the string r , say, at position pos . Then the string $r_1 r_2 \dots r_{pos}$ is a palindrome.

If there are at least three different characters in the string, then we choose one of them, x . Put the first occurrence of x at the beginning and all the others at the end. The rest of the characters will be placed between them in a sorted order: they are guaranteed not to form a palindrome. It is easy to see that this construction satisfies the condition of the problem.

Asymptotic $O(n)$.

Problem B. Breaking Bad

The idea is that most of the time, we should be able to get all possible remainders mod 5. Assume we have found a set of n cells, one in each row and column, such that there are 4 disjoint pairs of them with the following properties:

- Let these cells be $(r_1, c_1), (r_2, c_2)$. Then $a_{r_1, c_1} + a_{r_2, c_2} \not\equiv a_{r_1, c_2} + a_{r_2, c_1} \pmod{5}$.

Then we would have a number $1 \leq x_i \leq 4$ for each of these numbers, such that we can increase S by $x_i \pmod{5}$. It's easy to see that then we would be able to get all remainders. In fact, even the following more general statement is true:

- Let S be a set of $k < p$ nonzero remainders mod p , where p is a prime. Then there are at least $k + 1$ different remainders among subset sums of S (possibly empty).

So, if we can find such 4 disjoint pairs, we can just output **YYYYY**. Let's see how we can look for them and what happens if we don't succeed.

Assume there is such a pair of cells. Then there also exists a diagonally adjacent pair of cells like this. Indeed: if we had $a_{r,c} + a_{r+1,c+1} = a_{r,c+1} + a_{r+1,c} \pmod{5} \forall r, c$, we would have $a_{r,c} - a_{r,c+1} = a_{r+1,c} - a_{r+1,c+1} \pmod{5} \forall r, c \implies$ for a fixed c , $a_{r,c} - a_{r,c+1}$ is a constant mod 5. Then we can get that there exist arrays R, C such that $a_{r,c} = R_r + C_c$ for any r, c .

So, it's easy to find such a pair of cells if it exists. If we find one, remove the rows and columns with those cells from the grid, and start looking for the next one. If we found 4 such pairs, we won. Otherwise, we found at most 3.

The remaining part of the board has exactly the form described previously: $a_{r,c} = R_r + C_c$ for any r, c . Let's subtract $R_r + C_c$ from each cell: $\sum R_i + \sum C_i$ would be subtracted from each possible S , so our problem remains the same (almost). But now nonzero numbers can be present in at most 6 rows and 6 columns, let's say k rows and columns.

Here we can use your favorite bitmask algorithm. For each subset of those k rows, find all possible remainders you can get by select a cell in each of them outside of k columns. This takes $O(4^k \cdot n)$ with trivial dp, and can be sped up, but we don't care. Then repeat the same for columns. Finally, iterate over all possible choices of cells in the intersection of these k cells and columns. This would be around $O(k!)$.

Problem C. Chemistry Class

Sort the numbers, now $a_1 \leq a_2 \leq \dots \leq a_{2n}$.

Now, if $a_{2i-1} + A < a_{2i}$ for some i , then there is no solution. Indeed: there has to be at least one pair of form (a_x, a_y) with $x \leq 2i - 1, y \geq 2i$, since we can't split the first $2i - 1$ into pairs only among themselves. Otherwise, there is a solution: we can simply form pairs (a_{2i-1}, a_{2i}) . Now, assume $a_{2i-1} + A \geq a_{2i}$ for all i .

Consider solution with the largest number of pairs with distance at most B , denote it by k . We will refer to these k pairs as B -pairs, and to other pairs as A -pairs. First, note that we can make sure that no two A pairs "intersect" as segments, the same for B . Indeed, if elements from B pairs are $a_{x_1} \leq a_{x_2} \leq \dots \leq a_{x_{2k}}$, then we can form pairs $(a_{x_{2i-1}}, a_{x_{2i}})$.

Now, consider the first $2i$ elements. There are two cases:

- All of them are split into i pairs somehow. Then, we just need to find the maximal number of B -pairs we can form there separately.
- They are split into $i - 1$ pairs, with one A -element and one B -element remaining to be paired with elements with index $\geq i$, we refer to these pairs as "in-between pairs". Assume that when splitting only these $2i$ elements into i A -pairs and B -pairs, we can get at most k_1 B -pairs. Then we can have exactly k_1 B -pairs among these formed $i - 1$ pairs in this optimal solution. Indeed: if we have less than k_1 , then it's optimal to just break the in-between pairs, and pair the first $2i$ elements among themselves. This won't decrease the number of B -pairs. And, clearly, we can't have more than k_1 B -pairs. So, we will have exactly k_1 B -pairs.

One of these leftover elements is going to be a_{2i} . Our only goal is then to make the other leftover element as rightmost as possible.

Similarly, when we consider the first $2i - 1$ elements, we have all elements paired, except one leftover element. Our goal is once again to make it as rightmost as possible.

So, we can do a kind of dynamic programming. For even positions, $2i$, we can store the following values:

- The largest number of B -pairs we can form by splitting all $2i$ elements into pairs, k_1 .
- If we split them into $i - 1$ pairs, and a_{2i} belongs to in-between A -pair, what's the rightmost possible position of the remaining B -element (assuming the number of B -pairs is largest possible)
- If we split them into $i - 1$ pairs, and a_{2i} belongs to in-between B -pair, what's the rightmost possible position of the remaining A -element (assuming the number of B -pairs is largest possible)

For odd positions, we will keep similar things. The transitions will take $O(1)$ time, and the total asymptotic is $O(n)$.

Problem D. Daily Disinfection

The operation that we can do is just the following:

- Swap adjacent 0 and 1

We need to find the smallest number of operations required so that every position has been 0 at some point.

Here is another rephrasing of the problem. We have several 0s. In one operation, we can move 0 to any adjacent position. We need to visit all positions with 0s in the smallest number of operations.

The obvious lower bound is the number of 1s. In fact, it is achievable when any of the following is true:

- The leftmost character is 0
- The rightmost character is 0
- There are two adjacent 0s

Indeed, if the first condition holds, then each 0 can just "visit" the ones to the right of it (before the next 0 or the end of the string). Same for the second case. For the third case, if these zeros are at positions $i, i + 1$, then all zeros at positions $\leq i$ can visit ones to the left of them, and all zeros at positions $\geq i + 1$ can visit ones to the right of them.

The remaining case is trickier. It turns out that the minimal number of operations required would be equal to the number of 1s plus the length of the shortest segment of ones. The proof is left to the reader as an exercise.

Total asymptotic: $O(n)$.

Problem E. Equalizer Ehrmantraut

If for each i $a_i = b_i$, then the conditions of the problem are trivially satisfied. Otherwise, consider the smallest x for which there exists position pos such that $a_{pos} \neq b_{pos}$, and $\min(a_{pos}, b_{pos}) = x$. Without loss of generality, $a_{pos} = x < b_{pos}$.

Consider an arbitrary position $i \neq pos$. We must have $\min(a_{pos}, b_i) = \min(b_{pos}, a_i)$. There are two ways this can happen:

- If $\min(a_{pos}, b_i) = \min(b_{pos}, a_i) = x$, then we have $a_i = x, b_i \geq x$.
- If $\min(a_{pos}, b_i) = \min(b_{pos}, a_i) < x$, then we have $a_i = b_i < x$.

Thus, for a fixed x , the array a is uniquely recoverable from the array b : if $b_i < x$, then $a_i = b_i$, otherwise $a_i = x$. At the same time, there must be at least one position pos where $b_i > x$. It's easy to see that such arrays will satisfy the problem.

Let's calculate the number of such arrays for a fixed x . It is equal to $m^n - x^n$: the total number of possible arrays b minus the number of arrays b with all elements not exceeding x .

Thus, the total answer is $m^n + 2 \cdot \sum_{x=1}^{n-1} (m^n - x^n)$.

The final runtime is $O(m \log n)$ if we count each power using fast exponentiation.

Problem F. Formal Fring

Let's understand when multiset S satisfies this condition. Assume that the sum of integers in multiset S has form $2^k + x$, where $0 \leq x < 2^k$. Note that if we can split it into two groups with the same highest bit, this bit must be $k - 1$. We will show that it can't be split into two groups with equal highest bit if and only if the following condition holds:

- There exists some bit $i \leq k$, such that the sum of powers of 2 which are at least 2^i in S is at least $2^{k+1} - 2^i$

If this condition holds, then no matter how we split, at least one group will contain at least half of that sum, and, since it's divisible by 2^i , it will contain at least 2^k . So, its highest bit would be k , and the highest bits won't be equal.

We can show that this condition is true by induction by the sum of elements of S . Denote it by n . It's trivial for $n = 1$.

Suppose that it's true for all smaller values, let's show it for n . Let S be any group we can't split this way, let $n = 2^k + x$ again. If $n = 2^{k+1} - 1$, the condition holds for $i = 0$. Now, consider all powers of 2 in S larger than 2^0 , and divide them by 2.

If we could split this group in the required way, we would be able to go back (multiply all powers of two by 2), and add the remaining ones into groups appropriately, and split the initial multiset. Indeed: if we have numbers a and b with equal highest bit, and need to add $n - a - b$ ones to them so that they still have equal highest bit, it's easy to do: always add 1 to the smaller number, and, unless $n = 2^{k+1} - 1$, we will get two numbers with same highest bit in the end.

So, we can't, and therefore there exists some $i \leq k - 1$ for which the condition holds there. This means that this condition holds for $i + 1$ in the initial multiset.

Now let's use this observation to solve the problem. First, we can calculate $\text{cnt}[i]$ = the total number of multisets with sum i for all $1 \leq i \leq n$. It's just:

- $\text{cnt}[i] = \text{cnt}[i - 1]$ for odd i
- $\text{cnt}[i] = \text{cnt}[i - 1] + \text{cnt}[i/2]$ for even i

That's because we can either get multiset for i by adding 2^0 to multiset for $i - 1$, or, if there are no ones, by multiplying all elements of multiset with sum $i/2$.

Now, we can calculate the answer. The answer looks like this:

- $\text{ans}[i] = 1$ for $i = 2^k$
- $\text{ans}[i] = \text{cnt}[i]$ for $i = 2^{k+1} - 1$
- For all other i :
 $\text{ans}[i] = \text{ans}[i - 1]$ for odd i
- $\text{ans}[i] = \text{ans}[i - 1] + \text{ans}[i/2]$ for even i

That's because for these other i , we can get a multiset in which this condition holds in the same two ways: by adding 2^0 to multiset for $i - 1$, or, if there are no ones, by multiplying all elements of multiset with sum $i/2$.

Total asymptotic: $O(n)$.

Problem G. Goodman

Let's divide the permutation into cycles: sequences of numbers $b_1, b_2, \dots, b_k$ such that $p_{b_1} = b_2, p_{b_2} = b_3, \dots, p_{b_k} = b_1$. It is easy to see that if the length of the cycle is greater than 1, then there is no permutation q for which the elements of the cycle in q would be in the same order as in the permutation $r = (p_{q_1}, p_{q_2}, \dots, p_{q_n})$. Indeed, the elements of the cycle are in the same positions in p and r , so if they are in the same order, we should have $q_i = p_{q_i}$ for each q_i in the cycle, but this is impossible.

Therefore, from each such cycle of length $k \geq 2$ we can take at most $k - 1$ elements in each common subsequence. We will show that this is achievable.

For each cycle one by one, we add to q $b_1, b_2, \dots, b_k$ in that order. To r we add $p_{b_1}, p_{b_2}, \dots, p_{b_k} = b_2, b_3, \dots, b_1$. After that, we can add the numbers $b_2, b_3, \dots, b_k$ to the common subsequence q and r ! (And in the case of $k = 1$, we add a single element b_1).

So, the solution is as follows: write out all the cycles in order. The total asymptotic is $O(n)$.

Problem H. Highway Hoax

Root tree from some node and consider subtree of node v with parent par . Let's look what happens when we do operation with nodes v, par :

- If v was S and par was F, then in subtree of v the number of S will decrease by 1
- If v was F and par was S, then in subtree of v the number of S will increase by 1

Note that the numbers of S in subtree of v never change during any other operations. From this, we can make the following observation: the number of S in subtree of every node v has only two values it can attain. Moreover, this value determines the direction of the edge from v to its parent.

It turns out that every configuration which satisfies these conditions is indeed reachable. The proof is left as an exercise to the reader. So, it's enough to count the number of possible final configurations of S and F, given that we know two possible values for the numbers of S for each subtree (these two values differ by 1, by the way). How to do it?

We can do tree dp . Let's say that for node v we know, there must be exactly x_v or $x_v + 1$ labels S inside its subtree. We will denote the numbers of ways to get corresponding configurations by $dp_0[v], dp_1[v]$ correspondingly.

Consider children of v : $v_1, v_2, \dots, v_k$. For each v_i , there are $dp_0[v_i]$ ways to get x_{v_i} labels S in its subtree, and $dp_1[v_i]$ ways to get $x_{v_i} + 1$. How to combine all this knowledge? With polynomials.

Indeed: consider polynomial $\prod_{i=1}^k (dp_0[v_i] \cdot t^{x_{v_i}} + dp_1[v_i] \cdot t^{x_{v_i}+1})$. Its coefficient near t^y represents the number of ways to assign labels in subtrees so that we have exactly y labels S. We can calculate this polynomial in $O(k \log k^2)$ with FFT and Divide and Conquer. Then, consider possible labels of node v , and update corresponding dp values.

The total asymptotic is $O(n \log n^2)$.

Problem I. Increasing Income

First, let's try to find some upper estimate on this sum. If $f((q_1, q_2, \dots, q_n)) + f((p_{q_1}, p_{q_2}, \dots, p_{q_n})) = n + k$ for some k , then there must be at least k positions pos such that $q_{pos} = \max(q_1, q_2, \dots, q_{pos})$ and $p_{q_{pos}} = \max(p_{q_1}, p_{q_2}, \dots, p_{q_{pos}})$. Note that these positions form an increasing subsequence in p : if these positions are $pos_1, pos_2, \dots, pos_k$, then we have:

- $q_{pos_1} < q_{pos_2} < \dots < q_{pos_k}$
- $p_{q_{pos_1}} < p_{q_{pos_2}} < \dots < p_{q_{pos_k}}$

Thus, the answer cannot exceed $n + LIS$, where LIS is the length of the longest increasing subsequence of the permutation p . We will show that $n + LIS$ is achievable.

To make it easier to understand the solution, let's look at the problem from a different angle: we have pairs (i, p_i) , and we want to arrange these pairs in some order to maximize the sum of the number of prefix maximums in the permutation of the first numbers from the pairs, and in the permutation of the second numbers in the pairs. For a pair a , we denote the first number in the pair as a^1 and the second as a^2 .

Consider a subsequence of increasing pairs of length LIS , arrange them in ascending order, let them be pairs $a_1, a_2, \dots, a_{pos}$. Our task is to arrange the remaining $n - LIS$ pairs so that the first or second number in each pair is a prefix maximum, and in the selected LIS pairs, each number in the pair is a prefix maximum.

For each other pair b , we will find the largest i such that $a_i^1 < b^1$ or $a_i^2 < b^2$. We will put the pair b between a_i and a_{i+1} , namely: first we will put all pairs b for which $b^1 > a_i^1$ in ascending order of b^1 , and then all pairs b for which $b^2 > a_i^2$. In each of these pairs, one of the elements will be a prefix maximum.

We can find LIS in $O(n \log n)$. The general asymptotic is $O(n \log n)$.

Problem J. Jesse's Job

Consider the split of the permutation into cycles. If there are at least two cycles, we can color one of them in yellow, and everything else in blue. This way, we would put all elements at their places.

Assume now there is only a single cycle. Then there is no way to put all elements at their places: this would mean that blue elements form cycle(s), and yellow too. Obviously, we can't get $n - 1$ elements at their places either.

It turns out, however, that we can get $n - 2$ elements at their places! Indeed, swap elements 1 and 2 of the permutation. It will decompose into two cycles. Color one cycle in yellow, one in blue. All elements would fall at the correct places, except 1 and 2 that we swapped. So, it's possible to get $n - 2$.

Asymptotics $O(n)$.

Problem K. Knocker

Obviously, if $a_i = a_j$ at the beginning, a_i will always be equal to a_j , so we will remove duplicates from the array. Also, without loss of generality, we can assume that $a_1 > a_2 > \dots > a_n$.

Suppose that after performing several operations, we got the array $[b_1, b_2, \dots, b_n]$. If $a_1 = b_1$, then we have never chosen $x \leq a_1$, and therefore no numbers have changed. Now suppose that $b_1 < a_1$. Let $a_1 - b_1 = y$.

Let us show that for all i such that $a_i \geq y$, $a_i - b_i = y$ holds. We show that for such i $a_1 - a_i$ does not change during the operations. Let's show it by induction.

Consider any step. Assume $a_1 - b_1 = y \leq a_i$ currently. Assume we mod by x , a_1 turns into a'_1 , and a_i turns into a'_i . Then $a_1 - a'_1 \leq a_1 - b_1 = y \leq a_i$, so a_i will decrease by at least $a_1 - a'_1$. From the other side, if a_i decreased by $a_i - a'_i$, a_1 also can decrease by this much. Hence, $a_1 - a'_1 = a_i - a'_i$. In addition, we still have $a'_1 - b_1 \leq a'_i$ (since both sides decreased by the same amount).

Thus, we must have $b_i = a_i - y$ for all i for which $a_i \geq y$. This is equivalent to applying the operation $x = y$ from the beginning. The only constraint is that we must have $y > \frac{a_1}{2}$, otherwise $a_1 \bmod y < a_i - y$. And then we simply solve the problem for all numbers less than y , with the only condition that we cannot choose $x \leq b_1$.

Now, the problem is easy to solve using dynamic programming: $dp[pos][lim]$ — the number of different configurations that can be obtained from the array $a[pos : n]$ if we can only choose $x \geq lim$. We can do the jumps in $O(\max(MAX, n))$ by simply iterating over the final value of b_{pos} .

The total runtime is $O(nMAX \cdot \max(n, MAX))$, where MAX denotes the maximum element in the array.

Comment: the author of the problem suspects that the dp can be optimized to $O(\max(n, MAX)^2)$, but leaves it as an exercise.

Problem L. Lalo's Lawyer Lost

For the tree, this problem is pretty well-known. The answer can be calculated as follows: for each edge, find the number of nodes on both sides and let those be x, y . Then, add $\min(x, y)$ to the answer. This is clearly an upper bound: this edge will be contained in at most $\min(x, y)$ paths between nodes from the same pairs. It can be shown that it's reachable: it's enough to choose a centroid, consider its subtrees, and pair nodes from different subtrees, which will always be possible.

Here, however, we have cycles. Still, we can try to use this idea. For edges that are not a part of any cycle, we can do the same: find the number of nodes on both sides: x, y , and add $\min(x, y)$ to the answer. Now, let's handle the edges of the cycles.

Consider cycle $v_1 - v_2 - \dots - v_k - v_1$. Let the subtrees of $v_1, v_2, \dots, v_k$ contain $x_1, x_2, \dots, x_k$ elements. The idea is pretty much the same: we want to maximize the contribution of this cycle into the total distances. For that, we want to pair up the nodes in these subtrees, so that the total distance travelled along the edges of this cycle over all the paths between these pairs is as large as possible. How do we pair them up optimally?

Well, if we pair up nodes from subtrees of v_i and v_j , the shortest path between them will contain $\min(|i - j|, k - |i - j|)$ edges of the cycle. Basically, we have the following problem now: we have n points on the circle: x_i at position i , and want to pair them up to maximize the sum of distances between them. How to do this?

Claim: it's optimal to pair opposite nodes. You can show this by simple exchange argument: if you paired nodes (a, b) and (c, d) with a, b, c, d going on circle in this order, you can change them to pairs $(a, c), (b, d)$, and the sum of distances won't decrease.

With this in mind, it's not hard to get the contribution of each cycle. Proving that it's achievable is left as an exercise for the reader.

Total asymptotic is then still just $O(n)$.